

Metodologii și Algoritmi pentru Generarea Cheilor Criptografice

Drăgan Marius-Cristian

11 mai 2026

Cuprins

1	Introducere	3
1.1	Context și relevanță	3
1.2	Obiective și delimitare	3
1.3	Metodologie	3
1.4	Structura lucrării	3
2	Fundamente ale aleatorului criptografic	5
2.1	Entropie și imprevizibilitate	5
2.2	Modele de adversar și criteriul Next-Bit	5
2.3	Extractori și pre-procesare	5
2.4	Distribuții statistice și independența biților	6
2.5	Moduri de eșec ale surselor de entropie	6
3	Generatoare de numere aleatoare și surse de entropie	7
3.1	Taxonomie și cerințe	7
3.1.1	Estimarea entropiei surselor fizice	7
3.1.2	Tabel comparativ al tipurilor de generatoare	8
3.2	Surse fizice și modelare	8
3.3	DRBG și standardizarea NIST	8
3.3.1	Hash_DRBG	8
3.3.2	HMAC_DRBG	9
3.3.3	CTR_DRBG	9
3.4	Integrarea în sisteme de operare	9
3.5	Testare operațională și monitorizare	9
3.5.1	Health tests conform NIST SP 800-90B	10
3.6	Zgomotul cuantic și generatoarele QRNG	10
3.6.1	Principii fizice în QRNG	10
3.6.2	Avantaje față de TRNG-urile clasice	11
3.7	Entropy Pooling și managementul în sistemele de operare	11
3.7.1	Arhitectura nucleului Linux: /dev/random și /dev/urandom	11
3.7.2	Estimarea entropiei și blocarea (Blocking vs. Non-blocking)	11
3.7.3	Problema boot-ării (Boot-time Entropy)	12
3.7.4	Entropy-as-a-Service în medii virtualizate	12
4	Algoritmi de generare a cheilor	13
4.1	Chei simetrice	13
4.1.1	Programa de chei (Key Schedule) a AES	13
4.2	RSA: generarea primelor	13
4.2.1	Optimizări și dimensiuni recomandate pentru RSA	14

4.3	ECC: selectarea scalarului privat	14
4.3.1	Parametri de curbă standardizați	15
4.4	Nonces și semnături	15
4.5	Funcții de derivare a cheilor (KDF)	15
4.5.1	Argon2 – protecția bazată pe memoria intensivă	15
4.6	Managementul și ciclul de viață al cheilor	16
4.6.1	Stocarea și izolarea cheilor private	16
4.6.2	Rotația și expirarea cheilor	17
5	Vulnerabilități și Atacuri Istorice asupra Generării Cheilor	18
5.1	Incidentul Debian OpenSSL (2008)	18
5.2	Atacul asupra noncelor în ECDSA (Sony PS3 și Bitcoin)	18
5.3	Dual_EC_DRBG și suspiciunea de Backdoor	19
5.4	Atacuri prin injectarea de defecte (Fault Injection)	19
5.5	Atacul asupra Java RNG în Android 4.2	19
5.6	Mitigări și recomandări pentru practicieni	20
5.7	Analiza comparativă a spațiului de căutare	20
5.8	Analiza atacurilor asupra generatoarelor hardware	21
6	Evaluare și studii de caz	22
6.1	Bateria de teste statistice NIST SP 800-22	22
6.1.1	Interpretarea rezultatelor	23
6.1.2	Exemplu practic: testarea unui generator bazat pe ChaCha20 . . .	23
6.2	Studiu de caz: Cloudflare LavaRand	24
6.2.1	Implementarea din sediul San Francisco	24
6.2.2	Diversificarea surselor	25
6.3	Studiu de caz: Intel RDRAND și agregarea surselor hardware	25
7	Concluzii și direcții viitoare	26

Capitolul 1

Introducere

1.1 Context și relevanță

În securitatea informației, criptografia este mecanismul care asigură confidențialitatea, integritatea și autentificarea. Totuși, oricât de sofisticat ar fi un algoritm criptografic, securitatea sa reală depinde direct de calitatea cheilor utilizate. O cheie generată cu entropie insuficientă reduce semnificativ spațiul de căutare și permite atacuri de predicție sau enumerare, chiar dacă algoritmul rămâne matematic corect.

1.2 Obiective și delimitare

Lucrarea își propune să analizeze în profunzime procesul de generare a cheilor criptografice, punând accent pe:

- modelarea entropiei și evaluarea imprevizibilității;
- mecanismele și algoritmii pentru generarea și derivarea cheilor;
- criteriile de testare și validare a generatoarelor.

Se evită o fragmentare excesivă a subiectului, preferând dezvoltarea în profunzime a câtorva secțiuni centrale.

1.3 Metodologie

Analiza combină studiu teoretic, sinteză din standarde (NIST SP 800-90 [1] și SP 800-22 [2]) și descrierea algoritmilor utilizați în practică. Se includ exemple de algoritmi în pseudocod, tabele comparative și o schemă conceptuală a fluxului de generare a cheilor.

1.4 Structura lucrării

Lucrarea este organizată în șase capitole, fiecare abordând o dimensiune esențială a procesului de generare a cheilor criptografice:

- **Capitolul 2** fundamentează cadrul teoretic al aleatorului criptografic, incluzând entropia, modelele de adversar și tehnicile de prelucrare a surselor brute.

- **Capitolul 3** prezintă generatoarele de numere aleatoare – TRNG, PRNG, CSPRNG – împreună cu standardizarea NIST și sursele de entropie hardware și software.
- **Capitolul 4** detaliază algoritmi specifici sistemelor simetrice (AES) și asimetrice (RSA, ECC), precum și funcțiile de derivare a cheilor (KDF).
- **Capitolul 5** analizează vulnerabilități istorice și atacuri reale asupra generării cheilor, extrăgând lecții utile pentru practicieni.
- **Capitolul 6** prezintă metodologii de evaluare statistică (NIST SP 800-22) și studii de caz industriale (Cloudflare LavaRand, Intel RDRAND).

Capitolul 2

Fundamente ale aleatorului criptografic

2.1 Entropie și imprevizibilitate

Entropia lui Shannon măsoară incertitudinea unei variabile aleatoare:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i) \quad (2.1)$$

În criptografie este relevantă și entropia minimă, care consideră cel mai probabil rezultat:

$$H_{\infty}(X) = - \log_2 \left(\max_i P(x_i) \right) \quad (2.2)$$

Aceasta este o măsură conservatoare, utilă pentru evaluarea surselor reale de entropie, unde bias-ul poate fi exploatat de adversari.

2.2 Modele de adversar și criteriul Next-Bit

Un generator este considerat securizat dacă un adversar, cunoscând o secvență de biți, nu poate prezice bitul următor cu probabilitate semnificativ mai mare decât $1/2$. Acest criteriu formalizează noțiunea de aleator criptografic și impune cerințe stricte asupra designului generatorului.

2.3 Extractori și pre-procesare

Sursele fizice produc de obicei biți cu distribuție imperfectă. Pentru a obține o secvență aproape uniformă se folosesc funcții de dispersie:

$$Z = \text{Hash}(X \parallel \text{salt}) \quad (2.3)$$

Acest pas reduce bias-ul și combină mai multe surse independente, crescând entropia efectivă.

2.4 Distribuții statistice și independența biților

O sursă aleatoare ideală produce biți independenți și identic distribuiți (i.i.d. – independent and identically distributed). În practică, independența se verifică prin analiza corelațiilor statistice:

$$\rho(k) = \frac{E[(X_i - \mu)(X_{i+k} - \mu)]}{\sigma^2} \quad (2.4)$$

Coefficientul de autocorelație $\rho(k)$ măsoară dependența liniară între biți distanțați cu k poziții. Pentru un șir cu adevărat aleator, $\rho(k) \approx 0$ pentru orice $k \neq 0$. Testele *Runs* și *Serial* din suita NIST SP 800-22 detectează abateri sistematice de la independență.

2.5 Moduri de eșec ale surselor de entropie

Sursele fizice de entropie pot eșua în mai multe moduri, fiecare cu impact diferit asupra securității:

- **Eșec catastrofic:** Sursa încetează complet să producă biți (de exemplu, deconectarea unui senzor hardware). Detectabil imediat prin health tests.
- **Bias constant:** Distribuția devine asimetrică (de exemplu, 70% biți de 1). Reducerea entropiei minime este cuantificabilă.
- **Corelație temporară:** Un eveniment extern (zgomot electromagnetic, variație de tensiune) introduce dependență între biți succesivi.

Capitolul 3

Generatoare de numere aleatoare și surse de entropie

3.1 Taxonomie și cerințe

Generatoarele se împart în TRNG (bazate pe fenomene fizice), PRNG (deterministice) și CSPRNG (PRNG securizate criptografic). Un CSPRNG robust trebuie să fie rezistent la compromiterea stării interne și să permită re-însămânțarea periodică.

3.1.1 Estimarea entropiei surselor fizice

Estimarea corectă a entropiei produse de o sursă fizică este una dintre cele mai dificile probleme în proiectarea generatoarelor. NIST SP 800-90B [3] definește o metodologie riguroasă de evaluare:

- **Testul de repetiție (Repetition Count Test):** Detectează dacă aceeași valoare apare consecutiv de prea multe ori, indicând o blocare a sursei.
- **Testul de adaptare (Adaptive Proportion Test):** Verifică dacă distribuția de frecvență a valorilor rămâne consistentă pe măsură ce numărul de eșantioane colectate crește.
- **Estimarea entropiei minime pe coloane:** Pentru surse care produc cuvinte de k biți, se analizează independent fiecare poziție de bit, iar entropia totală este suma entropiilor minime pe coloană.

O limită inferioară conservatoare a entropiei disponibile se calculează cu formula:

$$H_{estimate} = -\log_2 \left(\max_{v \in V} \frac{count(v)}{N} \right) \quad (3.1)$$

unde V este mulțimea valorilor posibile, $count(v)$ frecvența valorii v , iar N numărul total de observații.

3.1.2 Tabel comparativ al tipurilor de generatoare

Tabela 3.1: Comparație între tipurile de generatoare de numere aleatoare

Proprietate	TRNG	PRNG	CSPRNG
Sursa de entropie	Fizică	Deterministică	Deterministică
Viteză de generare	Scăzută (~Mbps)	Foarte ridicată (~Gbps)	Ridică (~Gbps)
Reproducibilitate	Nu (fizic)	Da (același seed)	Nu (seed secret)
Securitate criptografică	Da (intrinsec)	Nu (predictibil)	Da (bazată pe seed)
Necesită însămânțare	Nu	Da	Da
Cost hardware	Ridicat	Zero	Zero
Exemplu	Zgomot termic	rand() în C	/dev/urandom

3.2 Surse fizice și modelare

Sursele fizice includ zgomot termic, jitter de ceas și efectul fotoelectric. Pentru fiecare sursă se evaluează stabilitatea, independența și rezistența la manipulare. În practică, implementările moderne agregă mai multe surse pentru a reduce riscul ca o singură sursă să fie compromisă.

3.3 DRBG și standardizarea NIST

NIST definește DRBG-uri bazate pe funcții hash, HMAC sau cifruri bloc [1]. Un DRBG are trei etape principale: inițializare cu entropie, generare deterministă și re-însămânțare. Re-însămânțarea limitează fereastra de expunere în cazul compromiterii stării interne.

Flux conceptual al generării cheilor

Surse fizice de entropie → Pre-procesare (hash/extractor) → DRBG/CS-PRNG → KDF → Cheie criptografică

Figura 3.1: Fluxul simplificat al procesului de generare a cheilor

3.3.1 Hash_DRBG

Primul DRBG standardizat utilizează o funcție hash criptografică (SHA-256 sau SHA-512) pentru a genera biți. Starea internă include o valoare V (seed-ul curent) și o constantă C :

```
Input: requested_bits, V, C, seed_len
Output: sir de biti aleatori
1. m = ceil(requested_bits / hash_len)
2. data = V
3. Pentru i = 1..m:
4.   block(i) = hash(data)
5.   data = data + 1
6. output = block(1) || block(2) || ... || block(m)
7. V = (V + C + 1) mod (2^seed_len)
8. Returneaza primii requested_bits din output
```

3.3.2 HMAC_DRBG

HMAC_DRBG utilizează HMAC în locul funcției hash directe, oferind o separare mai clară între fazele de însămânțare, generare și re-însămânțare. Starea internă constă din cheia K și variabila internă V :

```
Input: provided_data, K, V
Output: K_nou, V_nou
1. K = HMAC(K, V || 0x00 || provided_data)
2. V = HMAC(K, V)
3. Daca provided_data != NULL:
4.   K = HMAC(K, V || 0x01 || provided_data)
5.   V = HMAC(K, V)
6. Returneaza (K, V)
```

Listing 3.2: HMAC_DRBG – actualizarea stării interne

3.3.3 CTR_DRBG

CTR_DRBG utilizează un cifru bloc (AES) în modul CTR. Este cel mai rapid dintre cele trei variante NIST, dar necesită o implementare corectă a cifrului bloc subiacent:

```
Input: requested_bits, V, key
Output: blocuri criptate (biti)
1. temp = ""
2. Cat timp lungime(temp) < requested_bits:
3.   V_out = AES_encrypt(key, V)
4.   temp = temp || V_out
5.   V = V + 1
6. output = primii requested_bits din temp
7. key = AES_encrypt(key, V + 1)
8. Returneaza output
```

Listing 3.3: CTR_DRBG – generarea biților

3.4 Integrarea în sisteme de operare

Sistemele moderne oferă API-uri securizate pentru aleator: `/dev/urandom` și `getrandom()` în Linux, `BCryptGenRandom` în Windows. Utilizarea acestora este recomandată în locul implementărilor proprii, deoarece includ validări și colectare automată de entropie.

3.5 Testare operațională și monitorizare

Pe lângă testele statistice, generatoarele trebuie monitorizate în timp real prin health tests care detectează blocaje, bias-uri persistente și atacuri de injecție. Aceste teste sunt cerințe explicite în standardele moderne.

3.5.1 Health tests conform NIST SP 800-90B

Standardul NIST SP 800-90B [3] definește două teste de sănătate care rulează continuu în fundal:

- **Repetition Count Test:** Semnalizează o eroare dacă aceeași valoare (același cuvânt de n biți) apare de C ori consecutiv. Pragul C se calculează pe baza entropiei minime declarate a sursei:

$$C_{threshold} = \left\lceil \frac{-\log_2(\alpha)}{\log_2(1 - H_{min}/word_size)} \right\rceil \quad (3.2)$$

unde α este probabilitatea de alarmă falsă (de obicei 2^{-40}).

- **Adaptive Proportion Test:** Monitorizează proporția unei anumite valori într-o fereastră glisantă de W eşantioane. Dacă frecvența observată depășește un prag, se semnalizează o eroare. Acest test detectează bias-uri persistente care nu sunt vizibile în testul de repetiție.

Aceste teste trebuie să ruleze înainte ca orice output al generatorului să fie eliberat către aplicație, iar în caz de eșec, generatorul trebuie să intre într-o stare de eroare (error state) din care nu poate fi folosit fără re-inițializare manuală.

3.6 Zgomotul cuantic și generatoarele QRNG

Spre deosebire de sursele de entropie clasice, care se bazează pe fenomene macroscopice deterministe dar complexe (haos), generatoarele de numere aleatoare cuantice (QRNG - Quantum Random Number Generators) exploatează natura intrinsec probabilistă a mecanicii cuantice. Aceasta oferă o garanție teoretică a aleatorului, deoarece rezultatul unei măsurători cuantice nu este doar greu de prezis, ci fundamental nedeterminat înainte de actul măsurării.

3.6.1 Principii fizice în QRNG

Cele mai comune implementări de QRNG utilizează proprietățile fotonilor, datorită ușurinței de manipulare a acestora la temperatura camerei:

- **Divizarea fasciculului (Beam Splitting):** Un foton incident pe o oglindă semi-transparentă are o probabilitate egală (50%) de a fi reflectat sau transmis. Detectoarele plasate pe cele două căi generează biți binari în funcție de calea aleasă de foton.
- **Timpul de sosire a fotonilor:** Se măsoară intervalul de timp dintre emisiile succesive de fotoni într-o sursă de intensitate scăzută. Deoarece dezintegrarea atomică și emisia sunt procese Poisson, intervalele de timp sunt variabile aleatoare independente.
- **Zgomotul de fază al laserului:** Exploatează fluctuațiile cuantice de fază într-un laser care operează aproape de pragul de emisie.

3.6.2 Avantaje față de TRNG-urile clasice

Spre deosebire de zgomotul termic ($k_B T$), care poate fi influențat de temperatura ambientală sau de interferențe electromagnetice externe (atacuri de injecție), zgomotul cuantic este imun la manipularea parametrilor clasici. Totuși, implementarea necesită o etapă riguroasă de *post-procesare* pentru a elimina bias-ul introdus de imperfecțiunile detectoarelor hardware (timp de reîncărcare, eficiență cuantică variabilă).

3.7 Entropy Pooling și managementul în sistemele de operare

Sistemele de operare moderne joacă rolul de „agregator” de entropie, colectând zgomot dintr-o multitudine de surse hardware heterogene și oferind o interfață unificată și securizată pentru aplicații.

3.7.1 Arhitectura nucleului Linux: `/dev/random` și `/dev/urandom`

Nucleul Linux menține un *entropy pool*, o structură de date care acumulează biți de entropie din evenimente de sistem impredictibile:

- **Întreruperi de tastatură și mouse:** Timpii exacti (la nivel de microsecunde) dintre apăsările de taste.
- **Zgomot de stocare (Disk I/O):** Variațiile în timpul de răspuns al căutării pe disc, influențate de turbulențele de aer din interiorul HDD-urilor (deși mai puțin eficiente pentru SSD-uri).
- **Network Jitter:** Variațiile în timpii de sosire a pachetelor de rețea.
- **Instrucțiuni CPU:** Utilizarea instrucțiunii `RDRAND` (disponibilă pe arhitecturile Intel/AMD) care accesează un TRNG hardware integrat pe cip.

3.7.2 Estimarea entropiei și blocarea (Blocking vs. Non-blocking)

O componentă critică a subsistemului de aleator este *estimatorul de entropie*. Acesta calculează o limită inferioară a biților de „prospețime” introduși în pool.

$$E_{pool} = \sum_{i=1}^n \min_entropy(S_i) \quad (3.3)$$

Din punct de vedere istoric, `/dev/random` era un dispozitiv „de blocare”: dacă estimatorul considera că entropia disponibilă este sub un anumit prag, procesul care solicita cheia era suspendat până la colectarea de noi evenimente. În schimb, `/dev/urandom` (unlimited random) utilizează un CSPRNG (bazat pe ChaCha20 în versiunile recente) care este însămânțat din pool-ul principal, oferind un flux continuu de date fără blocaje, fiind considerat suficient de sigur pentru majoritatea utilizărilor criptografice moderne (conforme cu recomandările NIST).

3.7.3 Problema boot-ării (Boot-time Entropy)

O vulnerabilitate majoră în sistemele embedded sau instanțele cloud este lipsa entropiei la pornirea sistemului [4]. Deoarece starea dispozitivului este predictibilă imediat după boot (lipsesc input-urile utilizatorului sau traficul de rețea), cheile generate în primele secunde pot fi vulnerabile. Soluțiile moderne includ stocarea unui *seed* pe disc la oprire și reîncărcarea acestuia la pornire sau utilizarea modulelor hardware de tip TPM (*Trusted Platform Module*).

3.7.4 Entropy-as-a-Service în medii virtualizate

În arhitecturile cloud-native, instanțele virtuale (containere, mașini virtuale) nu dispun de acces direct la hardware-ul fizic al gazdei. Pentru a rezolva această problemă, hipervizorii expun dispozitive virtuale de aleator:

- **virtio-rng:** Interfață standard în medii KVM/QEMU care redirecționează cererile de entropie de la mașina virtuală către un TRNG hardware partajat la nivel de gazdă fizică.
- **Microsoft Hyper-V:** Expune un generator de numere aleatoare securizat pentru mașinile virtuale Windows și Linux.
- **haveged (Hardware Volatile Entropy Gathering and Daemon):** Serviciu software care generează entropie pe baza jitter-ului de execuție al instrucțiunilor CPU și a variațiilor în timpii de acces la cache, fără a necesita hardware dedicat.

Capitolul 4

Algoritmi de generare a cheilor

4.1 Chei simetrice

În criptografia simetrică, cheia trebuie să fie uniform aleator. Pentru AES-128, spațiul cheilor este 2^{128} . Orice bias în generare reduce acest spațiu efectiv și scade securitatea practică.

4.1.1 Programa de chei (Key Schedule) a AES

După generarea cheii principale de 128, 192 sau 256 de biți (printr-un CSPRNG securizat), algoritmul AES derivă cheile de rundă prin intermediul programei de chei. Aceasta extinde cheia principală într-un set de chei de rundă, câte una pentru fiecare iterație a cifrului:

```
Input: key[16] (16 octeti - 128 biți)
Output: w[44] (44 cuvinte a câte 4 octeti)
1. Pentru i = 0..3:
2.   w[i] = key[4*i : 4*i+3]
3. Pentru i = 4..43:
4.   temp = w[i-1]
5.   Dacă i mod 4 == 0:
6.     temp = SubWord(RotWord(temp)) XOR Rcon[i/4]
7.   w[i] = w[i-4] XOR temp
8. Returnează w[0..43]
```

Listing 4.1: AES-128 Key Schedule

Operația SubWord aplică substituția S-Box pe fiecare octet al cuvântului, RotWord realizează o rotație ciclică la stânga cu un octet, iar Rcon este o constantă de rundă (round constant) predefinită. În AES-192 și AES-256, programele de chei sunt similare, dar cu numere diferite de cuvinte și de runde (12, respectiv 14).

4.2 RSA: generarea primelor

Generarea cheilor RSA se bazează pe selecția aleatoare a două numere prime mari [5], [6]. Procesul tipic include filtrare cu prime mici și testarea probabilistică Miller-Rabin. În continuare este prezentată o variantă simplificată:

```
Input: n (numar impar), t (iteratii)
Output: probabil prim sau compus
```

```

1. Scrie  $n-1 = 2^s \cdot d$ , cu  $d$  impar
2. Pentru  $i = 1..t$ :
3.    $a \leftarrow \text{aleator in } [2, n-2]$ 
4.    $x \leftarrow a^d \bmod n$ 
5.   Daca  $x = 1$  sau  $x = n-1$ , continua
6.   Repeta  $s-1$  ori:
7.      $x \leftarrow x^2 \bmod n$ 
8.     Daca  $x = n-1$ , continua la urmatoarea iteratie
9.   Daca nu s-a obtinut  $n-1$ , returneaza compus
10. Returneaza probabil prim

```

Listing 4.2: Testul Miller-Rabin (variantă simplificată)

4.2.1 Optimizări și dimensiuni recomandate pentru RSA

În practică, generarea primelor pentru RSA utilizează tehnici avansate de accelerare:

- **Filtrarea cu numere prime mici (sieving):** Înainte de a aplica testul Miller-Rabin, se testează divizibilitatea candidatului cu primele 500–1000 de numere prime. Aceasta elimină rapid majoritatea numerelor compuse fără a apela la testul costisitor.
- **Testul Baillie-PSW:** O combinație între Miller-Rabin cu baza 2 și testul Lucas cu parametri specifici. Nu se cunosc contra-exemple pentru numere mai mici de 2^{64} .
- **Generarea concurentă:** Se generează p și q în paralel (fire de execuție separate) pentru a reduce timpul total de generare a cheii.

```

1.  $p \leftarrow \text{prim aleator}$ ,  $q \leftarrow \text{prim aleator}$ ,  $p \neq q$ 
2.  $n \leftarrow p \cdot q$ 
3.  $\phi \leftarrow (p-1) \cdot (q-1)$ 
4. alege  $e$  cu  $\gcd(e, \phi) = 1$ 
5.  $d \leftarrow e^{-1} \bmod \phi$ 
6. cheia publica:  $(n, e)$ , cheia privata:  $d$ 

```

Listing 4.3: Generare chei RSA (schematic)

Tabela 4.1: Dimensiuni recomandate pentru chei RSA și niveluri de securitate

Securitate (biți)	Modul RSA	Status	Recomandare
80	1024 biți	Interzis (depășit)	–
112	2048 biți	Minim acceptat	Utilizabil
128	3072 biți	Recomandat (2025+)	Preferat
192	7680 biți	Înaltă securitate	Guvernamental
256	15360 biți	Foarte înaltă	Pregătire post-cuantică

4.3 ECC: selectarea scalarului privat

În ECC, cheia privată este un scalar aleator d în intervalul $[1, n-1]$, iar cheia publică este $Q = dG$ [5]. Deoarece spațiul de chei este mai mic decât la RSA, orice bias în alegerea

lui d are impact major. De aceea, CSPRNG-ul folosit trebuie să aibă entropie de înaltă calitate.

4.3.1 Parametri de curbă standardizați

Standardele NIST (P-256, P-384, P-521) și SECG (secp256k1) definesc curbe cu parametri prestabiliți: ordinul n , punctul generator G , coeficienții a și b ai ecuației $y^2 = x^3 + ax + b$, precum și câmpul finit de bază. Cheia privată d se generează uniform din intervalul $[1, n - 1]$.

Tabela 4.2: Parametri principali pentru curbe eliptice frecvent utilizate

Curba	Lungime cheie	Securitate (biți)	Utilizare principală
P-256	256 biți	128	TLS 1.3, FIPS
P-384	384 biți	192	Clasificat (guvernamental)
P-521	521 biți	256	Înaltă securitate
Curve25519	256 biți	128	X25519 (key exchange), Signal
secp256k1	256 biți	128	Bitcoin, Ethereum

4.4 Nonces și semnături

Schemele ECDSA și DSA necesită un nonce aleator pentru fiecare semnătură. Dacă nonce-ul este repetat sau predictibil, cheia privată poate fi recuperată. În practică se recomandă nonce determinist conform RFC 6979 [7] pentru a evita dependența de RNG în faza de semnare.

4.5 Funcții de derivare a cheilor (KDF)

KDF-urile transformă materiale secrete în chei robuste și separă cheile pe contexte diferite. Pentru parole se utilizează PBKDF2 [8], scrypt sau Argon2 [9], [10]; pentru chei brute se folosește HKDF [11].

```

Input: IKM (input key material), salt, info, L
1. PRK = HMAC(salt, IKM)
2. T(0) = ""
3. Pentru i = 1..ceil(L/HashLen):
4.   T(i) = HMAC(PRK, T(i-1) || info || i)
5. OKM = T(1) || T(2) || ...

```

Listing 4.4: HKDF (extract + expand)

4.5.1 Argon2 – protecția bazată pe memoria intensivă

Argon2, câștigătorul competiției Password Hashing Competition (2015), oferă rezistență la atacuri hardware masiv paralele (GPU, FPGA, ASIC) prin utilizarea intensivă a memoriei RAM. Parametrii principali de configurare sunt:

- **Time cost (t):** Numărul de iterații – costul computațional.

- **Memory cost (m):** Memoria utilizată, exprimată în KiB.
- **Parallelism (p):** Numărul de fire de execuție hardware.

```

Input: password, salt, t, m, p, tag_len
Output: tag (cheia derivata)
1. Aloca memorie: m * 1024 octeti
2. Init blocurile H0 si H1 din password si salt
3. Pentru i = 1..t:
4.   Pentru j = 0..m-1:
5.     index <- functie pseudo-aleatoare(seed, j, p)
6.     bloc_curent <- G(bloc_curent, memorie[index])
7.     memorie[j] <- bloc_curent
8. tag <- XOR ultimele tag_len blocuri
9. Returneaza tag

```

Listing 4.5: Argon2id – flux simplificat

Argon2d optimizează rezistența la atacuri side-channel, iar Argon2i este construit pentru a rezista atacurilor trade-off (memorie-timp). Varianta Argon2id combină ambele abordări și este recomandată pentru majoritatea aplicațiilor practice.

Tabela 4.3: Comparatie intre KDF-uri uzuale

KDF	Scop	Caracteristici
PBKDF2	Parole	Simplu, iteratii configurabile, rezistenta moderata la GPU
Argon2	Parole	Memorie intensiva, optim contra GPU/ASIC, parametri multipli
HKDF	Chei brute	Extract + expand, separare contextuala, folosit in TLS 1.3

4.6 Managementul și ciclul de viață al cheilor

Generarea cheilor este doar prima etapă dintr-un proces mai amplu, care include stocarea securizată, distribuția, rotația periodică și distrugerea controlată a materialului criptografic.

4.6.1 Stocarea și izolarea cheilor private

Cheile private trebuie protejate împotriva extragerii chiar și în scenariul în care atacatorul obține acces privilegiat la sistem. Soluțiile moderne includ:

- **HSM (Hardware Security Module):** Dispozitive hardware specializate care generează, stochează și execută operații criptografice fără a expune cheile în memoria principală a serverului [12]. HSM-urile sunt certificate conform standardelor FIPS 140-2 Level 3 sau 4.

- **TPM (Trusted Platform Module):** Cip integrat pe placa de bază a calculatoarelor moderne, capabil să genereze și să stocheze chei legate de configurația hardware specifică a dispozitivului.
- **Enclave securizate:** Tehnologii precum Intel SGX sau AMD SEV creează zone de memorie criptată, izolate complet de sistemul de operare gazdă și de alte procese.

4.6.2 Rotația și expirarea cheilor

Chiar și o cheie generată cu entropie maximă trebuie înlocuită periodic. Durata de viață recomandată depinde de contextul criptografic:

- **Chei de sesiune (TLS):** Durată de viață de ordinul orelor sau zilelor. În TLS 1.3, cheile de sesiune sunt derivate prin mecanismul de key update.
- **Chei de semnare:** Luni sau ani, în funcție de politica organizației și de nivelul de securitate cerut.
- **Chei rădăcină (Root CA):** 5–10 ani, stocate offline în medii securizate.

Rotația minimizează impactul unei compromiteri întârziate și limitează cantitatea de date protejate cu aceeași cheie.

Capitolul 5

Vulnerabilități și Atacuri Istorice asupra Generării Cheilor

Securitatea sistemelor criptografice moderne se bazează pe ipoteza că atacatorul nu poate ghici sau deduce cheia privată. Istoria securității informației demonstrează însă că implementarea defectuoasă a generatoarelor de numere aleatoare (RNG) a fost cauza principală a multor compromiteri majore, transformând algoritmi matematic siguri în sisteme triviale de spart.

5.1 Incidentul Debian OpenSSL (2008)

Unul dintre cele mai răsunătoare eșecuri din istoria criptografiei moderne a avut loc în distribuția Debian Linux [13]. Din dorința de a elimina avertismentele de tip „uninitialized variable” raportate de instrumente de analiză cod (precum Valgrind), un dezvoltator a comentat liniile de cod care adăugau entropie în procesul de generare a cheilor.

- **Efectul:** Singura sursă de entropie rămasă a fost ID-ul procesului (PID). Deoarece PID-urile au o valoare maximă limitată (32.768), numărul total de chei RSA care puteau fi generate a fost redus de la 2^{128} sau 2^{256} la doar câteva mii.
- **Consecință:** Atacatorii au putut pre-genera toate cheile posibile și să identifice cheia privată corespunzătoare oricărui certificat server sau chei SSH generate pe un sistem afectat într-un timp neglijabil.

5.2 Atacul asupra noncelor în ECDSA (Sony PS3 și Bitcoin)

Algoritmul de semnatura digitală cu curbe eliptice (ECDSA) necesită o valoare aleatoare unică, numită *nonce* (k), pentru fiecare semnătură generată.

Matematic, dacă un atacator obține două semnături diferite realizate cu aceeași cheie privată și același *nonce*, acesta poate recupera cheia privată prin algebră simplă:

$$d_A = \frac{(s_1 z_2 - s_2 z_1)}{(r s_2 - r s_1)} \pmod{n} \quad (5.1)$$

- **Cazul Sony PlayStation 3:** Sony a utilizat o valoare constantă pentru k în procesul de semnare a codului, permițând hackerilor să recupereze cheia master a consolei [14].
- **Portofele Bitcoin:** Diverse aplicații de tip portofel au utilizat generatoare de numere aleatoare slabe pe Android, ducând la generarea de nonce-uri predictibile și, implicit, la furtul fondurilor prin calcularea cheilor private de către atacatori care monitorizau blockchain-ul.

5.3 Dual_EC_DRBG și suspiciunea de Backdoor

Standardul NIST SP 800-90A includea inițial un generator numit Dual_EC_DRBG, bazat pe curbe eliptice. Ulterior, cercetătorii au demonstrat că acesta conținea o vulnerabilitate structurală care arăta ca un „backdoor” matematic [15].

- **Mecanismul:** Dacă un atacator cunoaște relația discret-logaritmică dintre două puncte pe curba eliptică utilizată în generator, acesta poate prezice toți biții viitori după ce observă doar 32 de octeți din output-ul generatorului.
- **Impact:** Acest caz a subliniat importanța transparenței în standardizarea algoritmilor și riscul utilizării unor constante „magice” a căror proveniență nu este clar explicată.

5.4 Atacuri prin injectarea de defecte (Fault Injection)

O metodă modernă de atac asupra generării cheilor nu vizează algoritmul, ci mediul hardware (TRNG). Prin manipularea tensiunii de alimentare sau a temperaturii (over-clocking extrem), un atacator poate forța un generator hardware să producă secvențe de biți constante (de exemplu, doar „1”).

Dacă sistemul nu dispune de *health tests* active (menționate în capitolul anterior), acesta va continua să genereze chei bazate pe acest input constant, producând chei identice sau extrem de slabe. Acest tip de atac este deosebit de periculos pentru dispozitivele de tip Smart Card și modulele HSM (*Hardware Security Modules*).

5.5 Atacul asupra Java RNG în Android 4.2

O altă vulnerabilitate notabilă a fost descoperită în implementarea RNG a platformei Android (versiunile 4.2–4.3). Generatorul `java.security.SecureRandom` suferea de o eroare de inițializare care compromitea complet securitatea cheilor generate [16].

- **Cauza tehnică:** Entropia colectată la pornirea dispozitivului era insuficientă, iar amestecarea (mixing) pool-ului nu se realiza corect. Ca urmare, dispozitive diferite puteau genera aceeași secvență de biți dacă erau pornite în același interval de timp.
- **Impactul practic:** Mii de portofele Bitcoin generate pe terminale Android au fost compromise, deoarece cheile private (ECDSA) erau identice pentru utilizatori

diferiți. Atacatorii care monitorizau blockchain-ul puteau identifica tranzacțiile și fura fondurile imediat ce acestea erau trimise către o adresă generată vulnerabil. Pierderile financiare totale au fost estimate la milioane de dolari.

- **Lecția învățată:** Această vulnerabilitate a determinat Google să revizuiască complet implementarea `SecureRandom` începând cu Android 4.4 (KitKat), introducând apeluri directe la subsistemul de aleator al nucleului Linux.

5.6 Mitigări și recomandări pentru practicieni

Pe baza incidentelor analizate în acest capitol, se pot formula următoarele recomandări practice pentru implementarea sigură a generării cheilor:

1. **Utilizarea API-urilor sistemului de operare:** Nu implementați niciodată un generator propriu de numere aleatoare criptografice. Folosiți `getrandom()` (Linux), `BCryptGenRandom` (Windows) sau `SecRandomCopyBytes` (iOS).
2. **Persistența semințelor la oprire:** Stocați un seed criptat pe disc la oprirea sistemului și reîncărcați-l la pornire pentru a evita problema lipsei de entropie în faza incipientă de boot.
3. **Health tests obligatorii:** Implementați teste de sănătate care rulează înainte de fiecare operație de generare și semnalizează imediat orice anomalie (bias constant, valori repetitive).
4. **Diversificarea surselor de entropie:** Combinați minim două surse independente de entropie (de exemplu, `RDRAND` hardware și întreruperi de sistem) pentru a elimina punctele unice de eșec.
5. **Audit extern periodic:** Solicitați evaluarea implementării de către terți specializați (laboratoare acreditate Common Criteria).
6. **Nonce-uri deterministe:** Pentru schemele de semnătură (ECDSA, EdDSA), utilizați nonce-uri deterministe conform RFC 6979 pentru a elimina complet riscul repetării nonce-ului din cauza unui RNG defectuos.

5.7 Analiza comparativă a spațiului de căutare

Tabelul de mai jos ilustrează reducerea securității în cazul eșecului generării entropiei:

Tabela 5.1: Impactul calității entropiei asupra securității

Scenariu	Entropie (biți)	Efort Brute-Force
Ideal (AES-128)	128	3.4×10^{38} încercări
Debian OpenSSL	15	32.768 încercări (secunde)
Nonce ECDSA repetat	0 (după prima obs.)	Instantaneu
PRNG neînsămânțat (C)	$\sim 10-20$	Minute pe un PC standard

5.8 Analiza atacurilor asupra generatoarelor hardware

Pe lângă atacurile software descrise, generatoarele hardware (TRNG) sunt vulnerabile la atacuri fizice avansate care vizează direct mediul de operare:

- **Injectarea de frecvență electromagnetică:** Prin suprapunerea unui semnal electromagnetic extern controlat, un atacator poate forța oscilatorul unui TRNG să sconteze la o frecvență diferită, reducând sau eliminând entropia generată.
- **Variația forțată a temperaturii:** Răcirea extremă sau încălzirea puternică a cristalelor de cuarț modifică proprietățile piezoelectrice ale acestora, putând bloca complet generarea de biți.
- **Atacuri optice (Optical Fault Injection):** Expunerea cipurilor siliconice la lumină laser intensă modifică starea tranzistorilor, permițând controlarea generării de biți în circuitele integrate.

Contramăsurile pentru aceste atacuri includ ecranarea electromagnetică (carcase metalice Faraday), senzori integrați de temperatură și lumină, precum și redundanța prin utilizarea mai multor oscilatoare independente a căror ieșire este combinată printr-o funcție de dispersie criptografică.

Capitolul 6

Evaluare și studii de caz

6.1 Bateria de teste statistice NIST SP 800-22

Pentru a valida un generator de numere aleatoare destinat utilizării criptografice, National Institute of Standards and Technology (NIST) a definit o suită de 15 teste statistice [2]. Acestea analizează diferite tipuri de non-aleatoriu care pot apărea într-o secvență binară.

Fiecare test returnează o valoare numită P – *valoare*. Dacă $P \geq 0.01$, secvența este considerată aleatoare cu un nivel de încredere de 99%. Dacă $P < 0.01$, secvența eșuează testul respectiv.

Tabela 6.1: Suita completă de teste statistice NIST SP 800-22

Nr.	Test	Obiectivul analizei statistice
1	Frequency (Monobit)	Proporția de 0 și 1 pe întreaga secvență.
2	Frequency within Block	Proporția de 1 în interiorul unor blocuri de m-biți.
3	Runs Test	Numărul total de rulări (secvențe neîntrerupte de biți identici).
4	Longest Run of Ones	Cea mai lungă secvență de 1 într-un bloc.
5	Binary Matrix Rank	Dependența liniară între sub-secvențe (matrice de rang).
6	Discrete Fourier Transform	Detectarea periodicității (vârfuri spectrale în semnal).
7	Non-overlapping Template	Frecvența de apariție a unor tipare predefinite (fără suprapunere).
8	Overlapping Template	Frecvența de apariție a unor tipare cu suprapunere parțială.
9	Maurer's Universal Statistical	Gradul de compresibilitate a secvenței (entropia reală).
10	Linear Complexity	Lungimea unui registru de feedback liniar (LFSR) care poate genera secvența.
11	Serial Test	Frecvența tuturor tiparelor posibile de n-biți (ex: 00, 01, 10, 11).
12	Approximate Entropy	Comparația frecvenței blocurilor suprapuse de două lungimi adiacente.
13	Cumulative Sums	Deviația mersului aleator (random walk) față de zero.
14	Random Excursions	Numărul de vizite într-o anumită stare în timpul unui mers aleator.
15	Random Excursions Variant	Analiza frecvenței vizitelor în stări specifice, comparativ cu distribuția ideală.

6.1.1 Interpretarea rezultatelor

Testarea nu se realizează pe un singur șir, ci pe sute de eșantioane (de exemplu, 100 de șiruri a câte 1 milion de biți). Validarea finală a generatorului presupune îndeplinirea a două criterii:

1. **Proporția de trecere:** Un procent minim de șiruri (de regulă peste 96%) trebuie să treacă fiecare test.
2. **Distribuția P-valorilor:** P-valorile obținute trebuie să fie distribuite uniform în intervalul $[0, 1]$. O acumulare de valori într-o anumită zonă indică un bias subtil, chiar dacă testele individuale sunt "trecute".

6.1.2 Exemplu practic: testarea unui generator bazat pe ChaCha20

Pentru a ilustra procesul de testare, considerăm un generator simplu bazat pe ChaCha20 (utilizat în Linux `/dev/urandom`). Un program de test care rulează suita NIST SP 800-22 ar putea avea următoarea structură pseudocod:


```

Input: generator, num_sequences, seq_length
Output: raport de testare
1. Pentru i = 1..num_sequences:
2.   seq[i] = generator.generate(seq_length)
3. Pentru fiecare test T in NIST_suite:
4.   pass_count = 0
5.   Pentru i = 1..num_sequences:
6.     p_value = T.execute(seq[i])
7.     Daca p_value >= 0.01:
8.       pass_count++
9.   proportion = pass_count / num_sequences
10.  Daca proportion < 0.96:
11.    Semnalizeaza FAIL pentru testul T
12.  Altfel:
13.    Semnalizeaza PASS pentru testul T
14. Returneaza raportul final

```

Listing 6.1: Testarea unui generator cu suita NIST SP 800-22

Un generator ChaCha20 bine implementat trece de obicei toate cele 15 teste cu proporții de succes între 98% și 100% pentru seturi de 100 de șiruri a câte 10^6 biți fiecare. Eșecul unui test indică fie o eroare de implementare a generatorului, fie o configurație incorectă a parametrilor de testare.

6.2 Studiu de caz: Cloudflare LavaRand

Unul dintre cele mai celebre și fascinante exemple de generare a entropiei la scară industrială este sistemul **LavaRand** [17], utilizat de compania Cloudflare pentru securizarea a aproximativ 10% din traficul total de internet.

6.2.1 Implementarea din sediul San Francisco

În recepția sediului principal Cloudflare din San Francisco se află un perete compus din aproximativ 100 de lămpi cu lavă. Deși lămpile cu lavă par a fi simple obiecte decorative, ele reprezintă o sursă critică de zgomot analogic. Haosul este generat de dinamica fluidelor sub acțiunea căldurii, un proces macroscopic care este practic imposibil de modelat matematic sau de prezis de către un atacator.

Sistemul funcționează după următorul flux tehnic:

- **Captura video:** O cameră de înaltă rezoluție filmează în permanență peretele de lămpi.
- **Digitizarea zgomotului:** Imaginea capturată este influențată nu doar de mișcarea bulelor de lavă, ci și de variațiile luminii ambientale, trecerea persoanelor prin recepție și zgomotul electronic al senzorului camerei (ISO noise).
- **Hashing și Extracție:** Fluxul video este transformat într-o serie de octeți. Această serie brută (care are un bias inerent) este trecută printr-o funcție de dispersie criptografică (Hash), rezultatul fiind utilizat ca *seed* (sămânță) pentru CSPRNG-urile care generează cheile de sesiune TLS pentru serverele companiei.

6.2.2 Diversificarea surselor

Cloudflare nu se bazează exclusiv pe lămpile din San Francisco. Pentru a asigura reziliența și a preveni un punct unic de eșec (Single Point of Failure), compania utilizează surse alternative în locații diferite:

- **Londra:** Un sistem bazat pe „haosul pendulului”, unde mișcarea a trei pendule duble este filmată și procesată digital.
- **Singapore:** Măsurători ale dezintegrării radioactive (zgomot cuantic) provenite de la surse de uraniu de intensitate extrem de mică.

Acest studiu de caz demonstrează o axiomă fundamentală în generarea cheilor: **diversitatea surselor de entropie**. Prin combinarea fenomenelor fizice macroscopice (lămpi cu lavă) cu cele cuantice și electronice, se obține un sistem în care un atacator ar trebui să compromită legile fizicii în mai multe locații geografice simultan pentru a putea prezice o singură cheie criptografică.

6.3 Studiu de caz: Intel RDRAND și agregarea surselor hardware

Procesoarele Intel moderne includ instrucțiunea **RDRAND**, care oferă acces la un TRNG hardware integrat pe cip. Acesta utilizează zgomotul termic al joncțiunilor de siliciu ca sursă de entropie și include un post-procesor intern bazat pe CBC-MAC.

Implementarea Intel RDRAND respectă standardele BSI (Germania) [18] și NIST SP 800-90A [1]. Totuși, utilizarea exclusivă a RDRAND a fost criticată [19] din cauza imposibilității de audit extern al implementării hardware. Din acest motiv, în sistemele securizate, output-ul RDRAND este amestecat cu entropie din alte surse (software) înainte de a fi utilizat pentru generarea de chei. Această practică, numită *entropy mixing*, asigură că niciun compromis hardware individual nu poate afecta securitatea finală. Un exemplu concret este utilizarea RDRAND ca intrare suplimentară în pool-ul de entropie al nucleului Linux, alături de întreruperile de la dispozitive periferice.

Capitolul 7

Concluzii și direcții viitoare

Generarea cheilor este un proces multidimensional în care matematica, hardware-ul și ingineria software sunt interdependente. Calitatea entropiei, alegerea algoritmilor și testarea continuă determină securitatea reală a sistemelor criptografice.

Direcții viitoare de cercetare includ:

- **Criptografia post-cuantică:** Algoritmii precum Kyber, Dilithium sau Falcon necesită generarea de chei cu proprietăți diferite față de schemele clasice, inclusiv distribuții Gaussiene discrete pentru unele scheme bazate pe rețele.
- **Verificarea formală a generatoarelor:** Utilizarea asistenților de demonstrare (Coq, Isabelle/HOL) pentru a verifica corectitudinea implementărilor de DRBG și a elimina erorile de programare la nivel formal.
- **Standardizarea QRNG:** Odată cu maturizarea tehnologiei cuantice, generatoarele QRNG vor deveni suficient de compacte și ieftine pentru a fi integrate ca periferice standard la nivel de System-on-Chip.

Pe lângă progresele tehnologice, cadrul legislativ și de reglementare joacă un rol din ce în ce mai important. Regulamente precum eIDAS (Europa) [20] și FIPS 140-3 (SUA) [21] impun cerințe stricte de certificare pentru modulele criptografice, inclusiv pentru generatoarele de numere aleatoare. Viitorul va aduce standarde mai stricte și audituri periodice obligatorii pentru orice sistem care procesează informații clasificate sau financiare.

Bibliografie

- [1] National Institute of Standards and Technology, „Recommendation for Random Number Generation Using Deterministic Random Bit Generators,” NIST, rap. teh. NIST SP 800-90A Rev. 1, 2015.
- [2] National Institute of Standards and Technology, „A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications,” NIST, rap. teh. NIST SP 800-22 Rev. 1a, 2010.
- [3] National Institute of Standards and Technology, „Recommendation for the Entropy Sources Used for Random Bit Generation,” NIST, rap. teh. NIST SP 800-90B, 2018.
- [4] T. Ts'o și S. Böhm, *Linux Random Number Generator – Kernel Documentation*, 2020. adresa: <https://www.kernel.org/doc/html/latest/admin-guide/random.html>.
- [5] A. Menezes, P. van Oorschot și S. Vanstone, *Handbook of Applied Cryptography*. CRC Press, 2001.
- [6] C. Paar și J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*. Springer, 2010.
- [7] T. Pornin, „Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA),” IETF, rap. teh. RFC 6979, 2013.
- [8] B. Kaliski, „PKCS #5: Password-Based Cryptography Specification Version 2.0,” IETF, rap. teh. RFC 2898, 2000.
- [9] A. Biryukov, D. Dinu și D. Khovratovich, „Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications,” în *IEEE European Symposium on Security and Privacy (EuroSP)*, IEEE, 2016.
- [10] A. Biryukov, D. Dinu și D. Khovratovich, „Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” IETF, rap. teh. RFC 9106, 2021.
- [11] H. Krawczyk și P. Eronen, „HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” IETF, rap. teh. RFC 5869, 2010.
- [12] N. Ferguson, B. Schneier și T. Kohno, *Cryptography Engineering: Design Principles and Practical Applications*. Wiley, 2010.
- [13] Debian Security Team, *Debian Security Advisory DSA-1571-1: OpenSSL – Predictable Random Number Generator*, 2008. adresa: <https://www.debian.org/security/2008/dsa-1571>.
- [14] fail0verflow, *Console Hacking 2011: PS3 Epic Fail*, 2011. adresa: <https://fail0verflow.com/blog/2011/ps3-epic-fail/>.

- [15] D. Shumow și N. Ferguson, „On the Possibility of a Back Door in the NIST SP 800-90 Dual EC PRNG,” în *CRYPTO 2007 Rump Session*, 2007.
- [16] Bitcoin.org, *Android Security Vulnerability*, 2013. adresa: <https://bitcoin.org/en/alert/2013-08-11-android>.
- [17] Cloudflare, *LavaRand in Production: The Nitty-Gritty Technical Details*, 2017. adresa: <https://blog.cloudflare.com/lavarand-in-production-the-nitty-gritty-technical-details/>.
- [18] Bundesamt für Sicherheit in der Informationstechnik, „AIS 31: Functionality Classes and Evaluation Methodology for True (Physical) Random Number Generators,” BSI, rap. teh., 2011.
- [19] I. Corporation, „Intel Digital Random Number Generator (DRNG) Software Implementation Guide,” rap. teh. Document 331936-003, 2018.
- [20] European Parliament, „Regulation (EU) No 910/2014 on Electronic Identification and Trust Services (eIDAS),” Official Journal of the European Union, rap. teh., 2014.
- [21] National Institute of Standards and Technology, „Security Requirements for Cryptographic Modules,” NIST, rap. teh. FIPS PUB 140-3, 2019.